Danielle Pauleen G. Labasa BSIT 2-2N Object-Oriented Programming Date:

1.) What is Java programming language? Discuss its key features and advantages.

Over the years, technology has become more advanced with microprocessors running inside tools that people use everyday. Consequently, these tools are becoming more interconnected despite having varied networking environments. Due to this, it poses a challenge for developers, because having different networking environments mean different hardware components and operating systems. So, in the world of distinctiveness, how will it be possible to apply functionality to one, which can still be applied to another without the need for modifications?

Java is a high-level programming language due to its nature to run on abstraction through the Java Virtual Machine (JVM), instead of relying on machine specific details. In relation to that, Java is designed to be platform-independent with the principle, "Write Once, Run Anywhere" or WORA. Instead of a computer manually installing extensions for compilation, the JVM is packed with the Java Runtime Environment (JRE), which makes it very portable and applicable on a wide range of devices. To add, the JVM makes performance and memory management more efficient due to its component, Just-IN-Time (JIT); this component optimizes execution by compiling frequently used bytecode into native machine code or library. Furthermore, Java supports multithreading, which allows multiple tasks to run simultaneously; this is made possible by the JVM's thread scheduler. Apart from that, JVM's bytecode verifier provides security for the code on a wide range of hardware. Additionally, Java can have an object-oriented nature, as it centers or represent real-world entities and combination of both data and behavior. This makes programs organized and scalable, hence useful for collaboration and larger systems. Lastly, Java's extensibility is supported by its Application Programming Interfaces (API) and Java Native Interface (JNI), as it bridges pre-written classes or similar libraries like C and C++.

2.) Explain the concept of platform independence in Java. How does Java achieve platform independence?

Signature

Syllf.

Danielle Pauleen G. Labasa BSIT 2-2M Object-Oriented Programming Date:

Platform independence means that the program can run on various operating systems without the need of modifying the original source code. Java achieves this platform independence by compiling the code into a universal format, byte-code with the use of the Java Virtual Machine (JVM). This bytecode can run on any system that has a compatible JVM installed. In relation to that, the JVM acts as an interpreter between the bytecode and the operating system. Each operating system has its own unique way of translating the universal bytecode into machine-specific instructions. Denoting that, each operating system has its own version of JVM—whether it be for Windows, mac OS, or Linux. Therefore, the same bytecode file can be executed consistently across any platform.

3.) Discuss the main components of a Java program and their roles.

A typical Java program consists of several components that work together: classes, variables, objects, packages, and import. Classes serve as blueprint for creating objects and methods, and defining data types. The entry point or main method to define the behavior of the program starts from within these classes. Following that, variables act as storage locations for data, which can be manipulated throughout the program; this could support different data types such as the primitive and reference types. Java also uses objects to represent real-world entities within the program. Through these, a Java program can access class methods and variables, which can be grouped with the use of packages. Packages group related classes together and avoid naming conflicts. Due to this, classes become more organized, making large-scale projects easier to navigate. Lastly, import statements promote modularity and efficiency, because it allows programmers to use predefined classes from the Java's standard library.

4.) What are the difference between JDK, JRE, and JVM in Java?

Signature

Sgnf.

Danielle Pauleen G. Labasa BSIT 2-2N Object-Oriented Programming Date:

Java runs in various operating systems because of its fundamental components or tools such as the Java Development Kit (JDK), Java Runtime Environment (JDE), and Java Virtual Machine (JVM). These tools can be explained and differentiated as follows:

* The Java Development kit (JDK) acts as an environment or platform for the programmer to develop and test a Java program. Moreover, it provides the programmer with the essential tools to efficiently and effectively build a Java program such as the javac and JRE. Javac compiles the java source code into class files; whilst the JRE translates the local Java bytecodes into its native language counterpart.
* The Jave Runtime Environment (JRE) loads and executes the Java program. It processes the transformed classes into objects and passes it through its many inner components.
* The Java Virtual Machine (JVM) ensures that the Java program runs regardless of the operating system it is on. It does this by translating the local Java bytecode into its corresponding native code.

In conclusion, the JDK encompasses both the JRE and JVM. It is the primary environment in which the programmer will utilize to develop and test a Java program. To access and run the program, the JRE will be called upon by the JDK to load and run the JVM, completing the Java program cycle.

5.) Explain the difference between interpreted and compiled languages. Is Java an interpreted or compiled language?

An interpreted programming language reads, compiles, and executes a source code line-by-line; it allows a programmer to easily troubleshoot and prototype the program. Meanwhile, a compiled programming language reads and compiles the entire source code before execution; this allow a faster development time,

Synf.

Danielle Pauleen G. Labara BSIT 2-2N Object-Oriented Programming Date:

as you can quickly compile and execute the entirety of the program.

Java is both and interpreted and compiled language. Java is a compiled language as it has its own compiler inside the Java Development Kit (JDK), called the javac, which compiles an entire java source code; the javac's primary function is to translate a class file into a form that can be read easily by the Java Virtual Machine (JVM), which is the bytecode - the machine language of JVM. The compiled class files are then loaded by the JVM's class loader, which are read and loaded as they are referenced inside the program - meaning, a specific class file will not be loaded into an object unless it is called upon via reference. Additionally, JVM reads the program line-by-line and processes it through the execution engine, which includes the Just-In-Time (JIT) compiler; this optimizes the next compilation of the same class files, ensuring that they run more efficiently. Due to this nature, Java is also essentially an interpreted languange. Java's property of being both an interpreted and compiled language is one of the key components that allows it to be platform-independent, secure, and a powerful language.

Signature